Лабораторна робота №4

MongoDB та Mongoose: міграція REST API на базу даних

Завдання 1: MongoDB Atlas та MongoDB Compass

Створено акаунт на cloud.mongodb.com та безкоштовний кластер M0 (AWS / Stockholm). У Network Access додано правило 0.0.0.0/0 для дозволу підключень з будь-якого IP. Створено Database User Parz з паролем. Підключено MongoDB Compass за рядком підключення.

Файл .env (не додається до репозиторію):

```
MONGODB_URI=mongodb://Parz:<password>@ac-yudw6v0-shard-00-
00.fzksie5.mongodb.net:27017,...
PORT=3000
```

Файл .env.example (шаблон без паролів):

```
MONGODB_URI=mongodb+srv://USER:PASSWORD@cluster.mongodb.net/recipes
PORT=3000
```

Завдання 2: Підключення Mongoose та оновлення server.ts

Файл: src/config/database.ts

```
import mongoose from 'mongoose';

export async function connectDB(): Promise<void> {
  const uri = process.env.MONGODB_URI ?? "mongodb://localhost:27017/recipes";
  await mongoose.connect(uri);
  console.log('MongoDB підключено');
}

export async function disconnectDB(): Promise<void> {
  await mongoose.connection.close();
}

mongoose.connection.on('error', (err) => { console.error('MongoDB помилка:', err); });
mongoose.connection.on('disconnected', () => { console.warn('Відключено'); });
```

Файл: src/server.ts — підключення до БД перед стартом сервера + graceful shutdown:

```
import 'dotenv/config';
import app from './app';
import { connectDB, disconnectDB } from './config/database';

async function start() {
  try {
    await connectDB();
    const server = app.listen(process.env.PORT ?? 3000, () => {
      console.log(`Сервер запущено на http://localhost:${PORT}`);
    });
    process.on('SIGTERM', async () => {
      server.close(async () => { await disconnectDB(); process.exit(0); });
    });
  } catch (err) {
    console.error(err); process.exit(1);
  }
}
start();
```

Завдання 3: Mongoose-схема та модель

Файл: src/models/recipe.model.ts

Реалізовано TypeScript-інтерфейс IRecipe, схему з валідацією, кастомний валідатор (цілочисельність), віртуальну властивість cookingTimeLabel.

```
import { Schema, model, Document } from 'mongoose';

export interface IRecipe {
  title: string;
  description?: string;
  cookingTimeMinutes: number;
  difficulty: 'easy' | 'medium' | 'hard';
  cuisine?: 'ukrainian' | 'italian' | 'asian' | 'american' | 'french' | 'other';
  servings?: number;
}

export interface IRecipeDocument extends IRecipe, Document {
  cookingTimeLabel: string; // virtual
  createdAt: Date;
  updatedAt: Date;
}

const recipeSchema = new Schema<IRecipeDocument>({
  title: { type: String, required: [true, "Назва є обов'язковою"], trim: true,
maxlength: 100 },
  cookingTimeMinutes: {
    type: Number, required: true, min: 1, max: 1440,
    // Кастомний валідатор — цілочисельність
    validate: { validator: (v: number) => Number.isInteger(v),
                message: "Час має бути цілим числом" }
```

```
  },
  difficulty: { type: String, required: true, enum: ['easy', 'medium', 'hard'] },
  cuisine: { type: String, enum: ['ukrainian', 'italian', 'asian', 'american',
'french', 'other'] },
  servings: { type: Number, min: 1, max: 100,
    validate: { validator: (v: number) => Number.isInteger(v), message: "Ціле число" }
},
}, { timestamps: true, toJSON: { virtuals: true }, toObject: { virtuals: true } });

// Віртуальна властивість — людиночитабельний час
recipeSchema.virtual("cookingTimeLabel").get(function () {
  const mins = this.cookingTimeMinutes;
  if (mins < 60) return `${mins} хв`;
  const h = Math.floor(mins / 60); const m = mins % 60;
  return m > 0 ? `${h} год ${m} хв` : `${h} год`;
});

export const RecipeModel = model<IRecipeDocument>('Recipe', recipeSchema);
```

Завдання 4: Заміна сховища на Mongoose

Файл: `src/storage/recipe.ts` — увесь Map-код замінено на виклики RecipeModel.

```
import { RecipeModel } from '../models/recipe.model';

export async function getAllRecipes(filters = {}, pagination = {}) {
  const query: Record<string, any> = {};
  if (filters.difficulty) query.difficulty = filters.difficulty;
  if (filters.cuisine) query.cuisine = filters.cuisine;
  if (filters.maxCookingTime) query.cookingTimeMinutes = { $lte: filters.maxCookingTime
};
  if (filters.search) query.title = { $regex: filters.search, $options: "i" };

  const page = pagination.page ?? 1; const limit = pagination.limit ?? 10;
  const [data, total] = await Promise.all([
    RecipeModel.find(query).sort(sortObj).skip((page-1)*limit).limit(limit),
    RecipeModel.countDocuments(query),
  ]);
  return { data, pagination: { page, limit, total, totalPages: Math.ceil(total/limit) }
};
}

export async function getRecipeById(id: string) { return RecipeModel.findById(id); }

export async function createRecipe(data) { return RecipeModel.create(data); }

export async function updateRecipe(id, data) {
  return RecipeModel.findByIdAndUpdate(id, data, { new: true, runValidators: true });
}

export async function deleteRecipe(id) { return RecipeModel.findByIdAndDelete(id); }
```

Завдання 5: Обробка помилок MongoDB

Файл: src/middleware/errorHandler.ts — розширено обробку специфічних помилок Mongoose та MongoDB.

```
import { ZodError } from 'zod';
import { Error as MongooseError } from 'mongoose';
import { MongoServerError } from 'mongodb';

export function errorHandler(err, _req, res, _next): void {
  // Zod — 400
  if (err instanceof ZodError) {
    res.status(400).json({ error: "Помилка валідації", details: err.issues });
    return;
  }
  // Mongoose ValidationError — 400
  if (err instanceof MongooseError.ValidationError) {
    res.status(400).json({ error: "Помилка валідації даних" });
    return;
  }
  // Mongoose CastError (невалідний ObjectId) — 400
  if (err instanceof MongooseError.CastError) {
    res.status(400).json({ error: "Невалідний формат ID" });
    return;
  }
  // Дублікат ключа — 409
  if (err instanceof MongoServerError && err.code === 11000) {
    res.status(409).json({ error: "Конфлікт: запис вже існує" });
    return;
  }
  res.status(500).json({ error: "Внутрішня помилка сервера" });
}
```

Завдання 6: Фільтрація, сортування та пагінація

GET /api/recipes тепер повертає об'єкт з полями data та pagination. Підтримуються параметри: ?difficulty, ?cuisine, ?maxCookingTime, ?search, ?page, ?limit, ?sort.

Сортування: ?sort=title (asc) або ?sort=-title (desc).

```
// Приклади запитів:
GET /api/recipes?difficulty=easy&page=1&limit=5
GET /api/recipes?cuisine=italian&sort=-cookingTimeMinutes
GET /api/recipes?maxCookingTime=30&search=pasta

// Відповідь:
{
  "data": [ { "_id": "...", "title": "Pasta", "cookingTimeLabel": "20 хв", ... } ],
  "pagination": { "page": 1, "limit": 5, "total": 12, "totalPages": 3 }
}
```

Спеціальний маршрут GET /api/recipes/quick повертає рецепти до 30 хвилин, відсортовані за часом приготування.

Завдання 7: Тестування з mongodb-memory-server

Файли: tests/globalSetup.ts, tests/globalTeardown.ts, tests/setup.ts, tests/recipe.test.ts

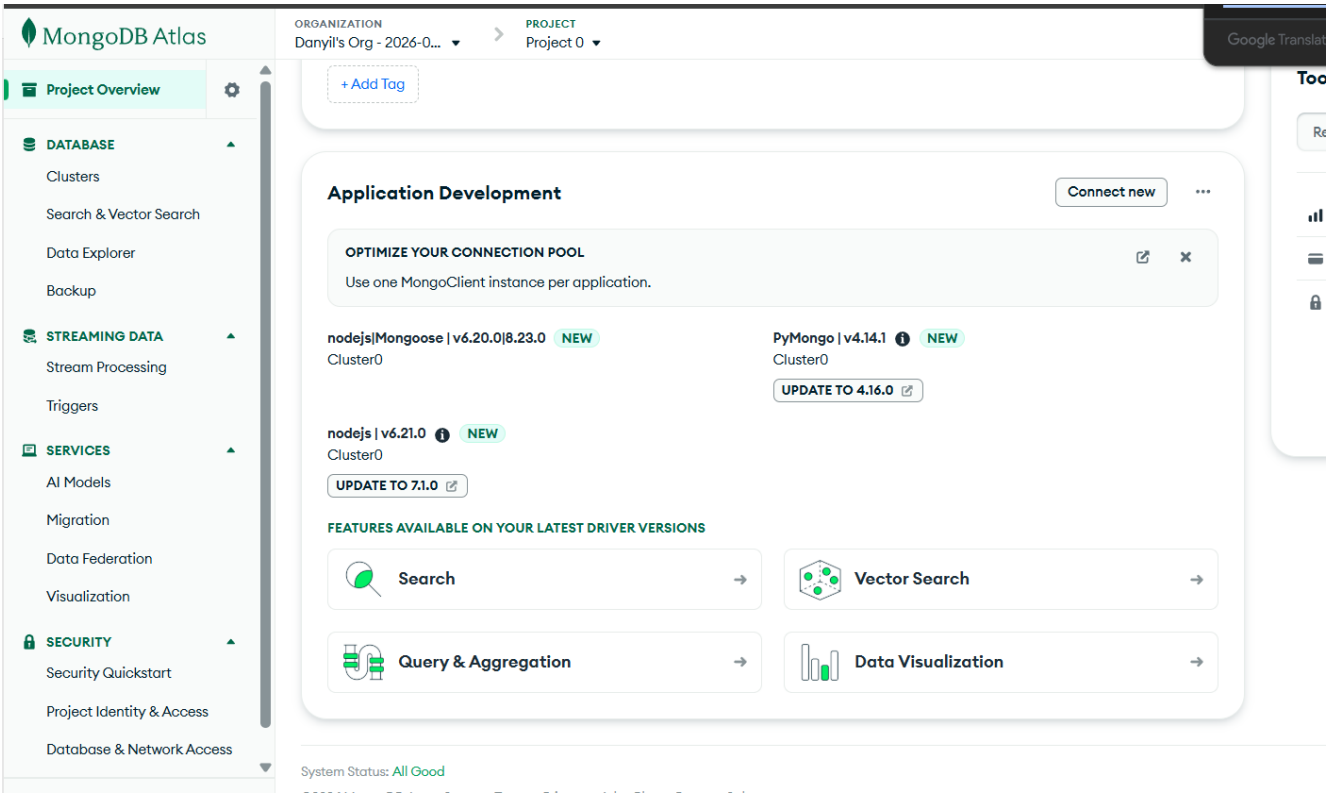
Написано 30 тестів: unit-тести моделі (8) + інтеграційні тести API (22). MongoMemoryServer запускається глобально через globalSetup/globalTeardown.

```
// tests/globalSetup.ts
import { MongoMemoryServer } from 'mongodb-memory-server';
export default async function globalSetup() {
  const mongod = await MongoMemoryServer.create();
  process.env.MONGODB_URI = mongod.getUri();
  (global as any).__MONGOD__ = mongod;
}

// tests/setup.ts
export async function setupTestDB() { await mongoose.connect(process.env.MONGODB_URI!);
}
export async function teardownTestDB() { await mongoose.connection.close(); }
export async function clearDB() {
  for (const key in mongoose.connection.collections) {
    await mongoose.connection.collections[key].deleteMany({});
  }
}

// tests/recipe.test.ts
beforeAll(async () => { await setupTestDB(); });
afterAll(async () => { await teardownTestDB(); });
afterEach(async () => { await clearDB(); });

describe('Recipe Model', () => {
  it('зберігає документ', async () => {
    const doc = await RecipeModel.create(validRecipe);
    expect(doc._id).toBeDefined();
  });
  it('virtual cookingTimeLabel: 90 хв -> 1 год 30 хв', async () => {
    const doc = await RecipeModel.create({ ...validRecipe, cookingTimeMinutes: 90 });
    expect(doc.cookingTimeLabel).toBe('1 год 30 хв');
  });
  // ... ще 6 unit-тестів моделі
});

describe('POST /api/recipes', () => { /* 6 тестів */ });
describe('GET /api/recipes', () => { /* 9 тестів */ });
describe('GET /api/recipes/:id', () => { /* 3 тести */ });
describe('PATCH /api/recipes/:id', () => { /* 4 тести */ });
describe('DELETE /api/recipes/:id', () => { /* 3 тести */ });
describe('GET /api/recipes/quick', () => { /* 2 тести */ });
```

Результати тестування

```
PS E:\2 курс\Node.Js\lab4_NodeJS> npm test
> jest --runInBand

Determining test suites to run...(node:7664) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
 PASS  tests/recipe.test.ts (32.078 s)
  Recipe Model
    √ зберігає документ з коректними полями (40 ms)
    √ автоматично виставляє createdAt та updatedAt (6 ms)
    √ virtual cookingTimeLabel повертає хвилини якщо < 60 (8 ms)
    √ virtual cookingTimeLabel повертає години та хвилини (6 ms)
    √ virtual cookingTimeLabel повертає цілі години (6 ms)
    √ не зберігає документ без title (8 ms)
    √ не зберігає документ з невалідним difficulty (3 ms)
    √ не зберігає документ з дробовим cookingTimeMinutes (4 ms)
  POST /api/recipes
    √ створює рецепт і повертає 201 (441 ms)
    √ повертає 400 якщо title відсутній (10 ms)
    √ повертає 400 якщо title порожній (11 ms)
    √ повертає 400 якщо difficulty невалідний (9 ms)
    √ повертає 400 якщо cookingTimeMinutes = 0 (11 ms)
    √ створює рецепт без опціональних полів (15 ms)
  GET /api/recipes
    √ повертає об'єкт з data та pagination (26 ms)
    √ повертає всі рецепти (30 ms)
    √ фільтрує за difficulty (26 ms)
    √ фільтрує за cuisine (24 ms)
    √ фільтрує за maxCookingTime (23 ms)
    √ фільтрує за search (регістронезалежно) (23 ms)
    √ комбінує кілька фільтрів (31 ms)
    √ підтримує пагінацію (41 ms)
    √ підтримує сортування за назвою (23 ms)
    √ підтримує сортування у зворотньому напрямку (25 ms)
  GET /api/recipes/:id
    √ повертає рецепт за id (15 ms)
    √ повертає 404 для неіснуючого валідного ObjectId (7 ms)
    √ повертає 400 для невалідного формату ID (9 ms)
  PATCH /api/recipes/:id
    √ оновлює рецепт (29 ms)
    √ повертає 404 для неіснуючого рецепту (8 ms)
    √ повертає 400 якщо дані невалідні (15 ms)
    √ повертає 400 для невалідного ID (7 ms)
  DELETE /api/recipes/:id
    √ видаляє рецепт і повертає 204 (24 ms)
    √ повертає 404 для неіснуючого рецепту (8 ms)
    √ повертає 400 для невалідного ID (7 ms)
  GET /api/recipes/quick
    √ повертає лише рецепти до 30 хвилин (35 ms)
    √ повертає порожній масив якщо немає швидких рецептів (14 ms)

Test Suites: 1 passed, 1 total
Tests:       36 passed, 36 total
Snapshots:   0 total
Time:        32.228 s
Ran all test suites.
PS E:\2 курс\Node.Js\lab4_NodeJS>
```

```
}
PS E:\2 курс\Node.Js\lab4_NodeJS> npm run dev

> lab4-recipes-api@1.0.0 dev
> ts-node src/server.ts

MongoDB підключено
Сервер запущено на http://localhost:3000
```



Висновок

У ході виконання лабораторної роботи було виконано міграцію REST API з in-memory Map-сховища на MongoDB з використанням Mongoose ODM. Налаштовано хмарний кластер MongoDB Atlas, підключення з graceful shutdown, Mongoose-схему з кастомним валідатором та віртуальною властивістю. Реалізовано фільтрацію через Mongoose Query API з операторами $lte та $regex, сортування та пагінацію. Для тестування використано mongodb-memory-server, що дозволяє запускати реальну MongoDB у пам'яті без зовнішніх залежностей.